Lidar Point Cloud Fusion and Filtering

1. Course Content

- Run the sample program and view the fused and filtered Lidar point cloud in RViz visualization.
- Understand the methods for Lidar point cloud fusion and filtering.

> [!NOTE]
>
> The case in this lesson uses the single-Lidar version of the vehicle as an example. All commands are consistent with the dual-Lidar version, with only the initial Lidar topic names differing.

2. View Lidar Point Cloud

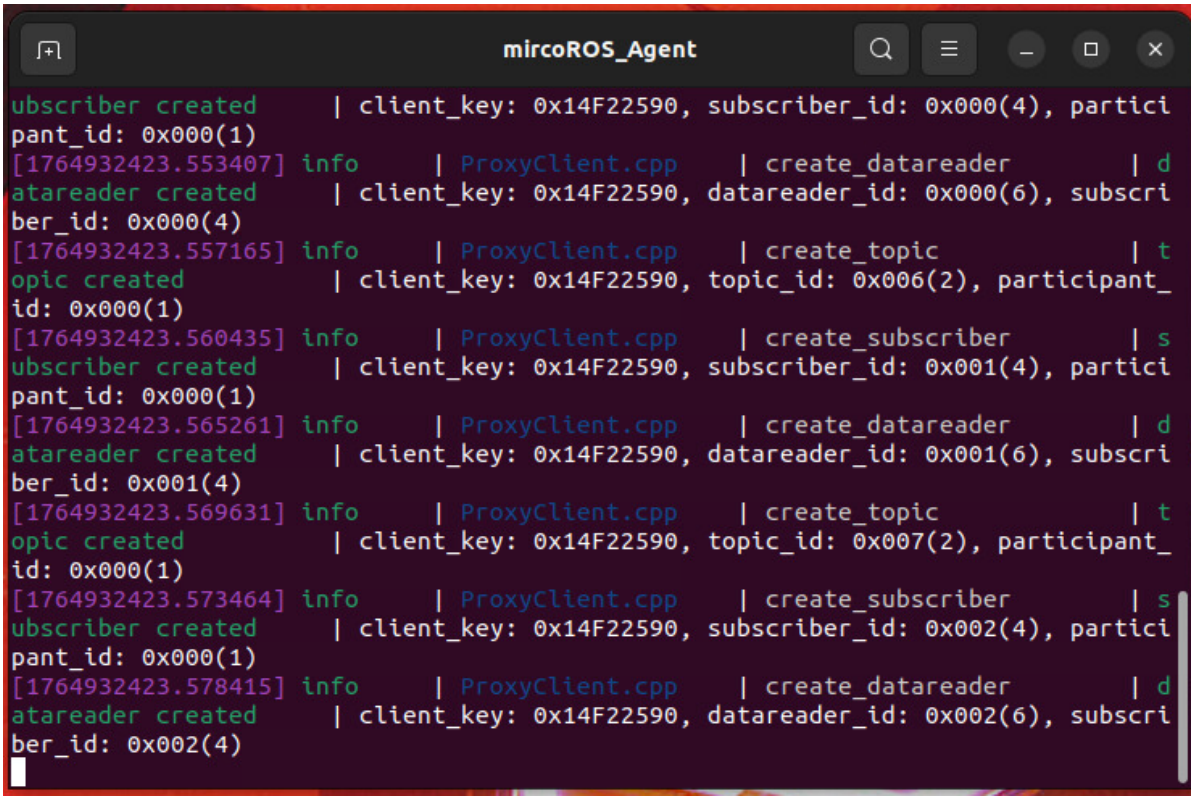
2.1 Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



2.2 Visualize Lidar Point Cloud

- Use RVIZ to visualize the Lidar's point cloud data.

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

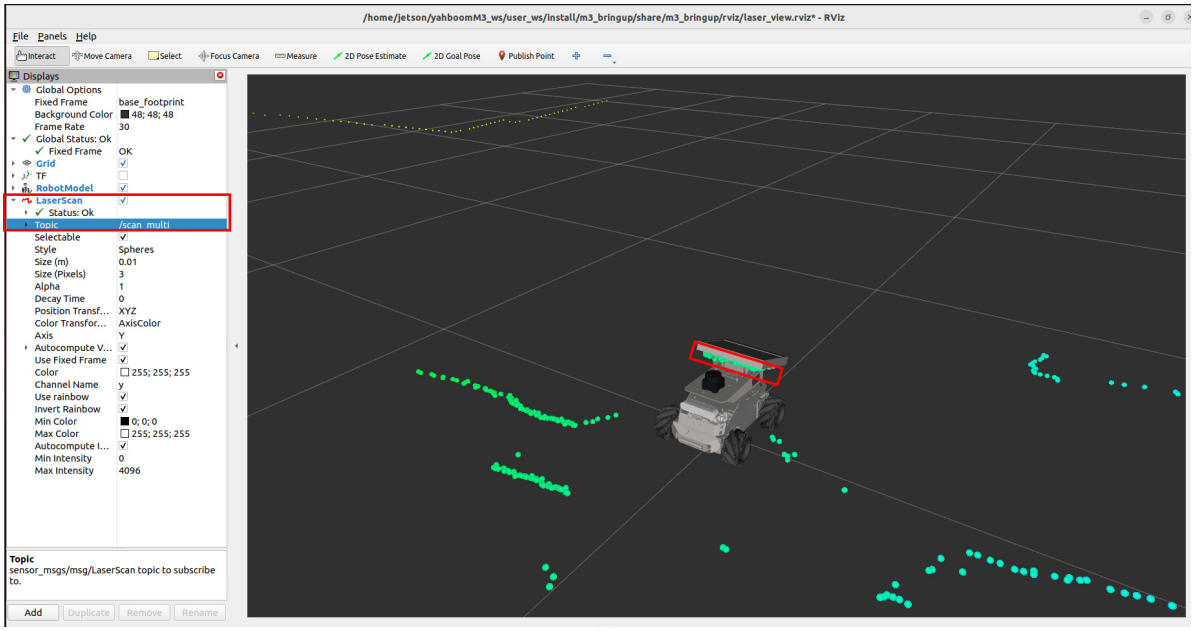
Open a terminal inside the container,

```
exec_m3
```

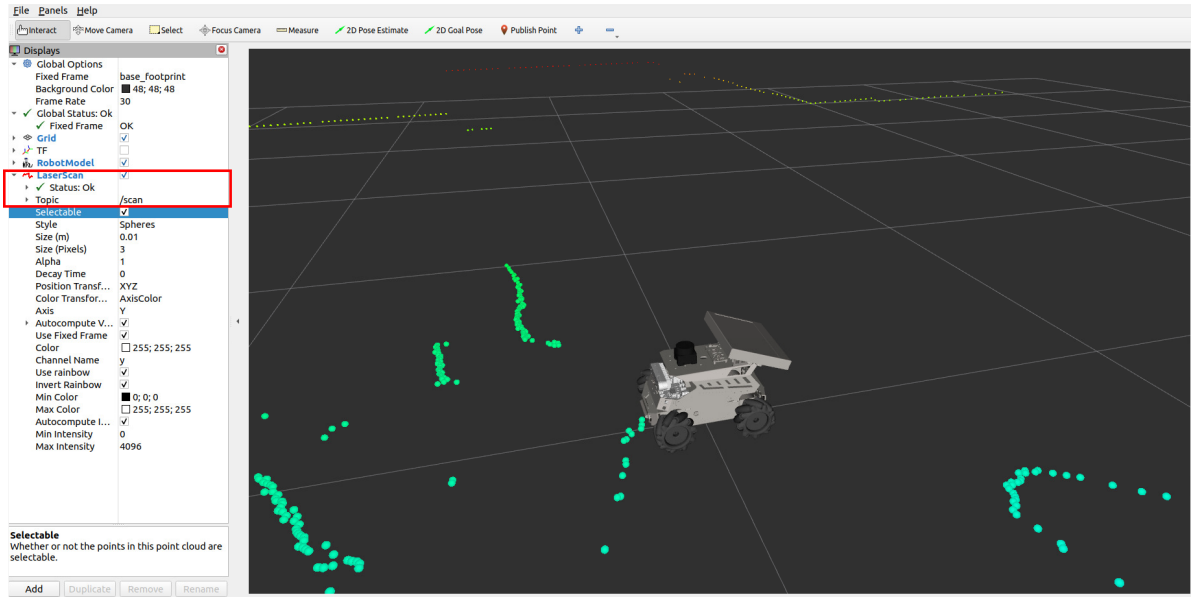
Enter the command,

```
ros2 launch m3_bringup laser_view.launch.py
```

- In the LaserScan options on the right, select the `Topic` and choose the `/scan_multi` topic. At this point, we see the fused point cloud information.
- However, this point cloud cannot be used directly because some points within the vehicle's body contour are useless. In SLAM mapping and navigation, these points would be treated as obstacles, interfering with the mapping and navigation functions.



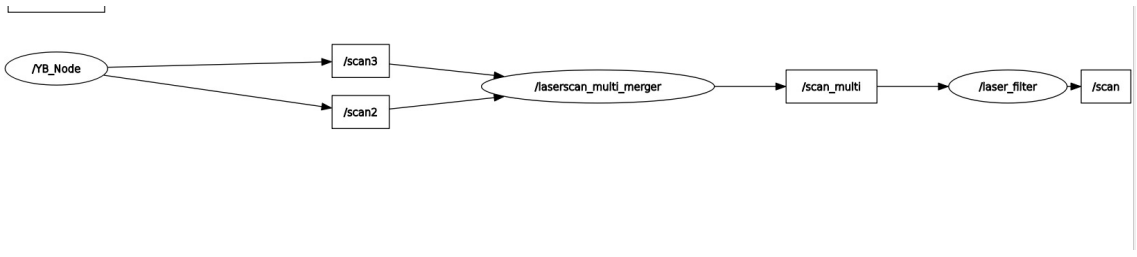
- Therefore, we need to filter out some of the point cloud data inside the vehicle's body. We change the `Topic` to `/scan`, which now displays the final filtered point cloud.



3. View Node Communication Graph

- Run the command in the terminal:

```
ros2 run rqt_graph rqt_graph
```



- The `/YB_node` is the chassis's `Micro-ROS` node. The `/scan2` and `/scan3` topics are the raw Lidar topics.
- The raw topics pass through the `/laserscan_multi_merger` node, which fuses the Lidar point clouds and publishes the fused point cloud topic `/scan_multi`.
- The `/laser_filter` node filters the fused point cloud `/scan_multi`, removing some points inside the vehicle's contour, and publishes the final `/scan` Lidar point cloud.

4. Source Code Analysis

4.1 Point Cloud Fusion

- Lidar point cloud fusion source code:

```
~/yahboomM3_ws/code_ws/src/laser_merge/src/laserscan_multi_merger.cpp
```

- The core program logic for point cloud fusion is as follows:

1. `scanCallback`: This is the callback function for subscribing to the Lidar `LaserScan` topic. It is responsible for converting single-Lidar scan data into a point cloud, performing coordinate system transformations, and, after all Lidar data has arrived, executing point cloud concatenation, which finally triggers the reconstruction of the laser scan data.
2. `pointcloud_to_laserscan`: This function converts the fused point cloud data back into a standardized `LaserScan` message and publishes it, while also retaining the ability to publish the point cloud.

```
    void LaserscanMerger::scanCallback(sensor_msgs::msg::LaserScan::SharedPtr
scan, std::string topic)
    {
        sensor_msgs::msg::PointCloud2 tmpCloud1, tmpCloud2;

        try
        {
            // Verify that TF knows how to transform from the received scan to
the destination scan frame
            tf_buffer_->lookupTransform(scan->header.frame_id.c_str(),
destination_frame.c_str(), scan->header.stamp, rclcpp::Duration(1, 0));
            projector_.transformLaserScanToPointCloud(scan->header.frame_id,
*scan, tmpCloud1, *tf_buffer_, range_max);
            pcl_ros::transformPointCloud(destination_frame.c_str(), tmpCloud1,
tmpCloud2, *tf_buffer_);
        }
        catch (tf2::TransformException &ex)
        {
            return;
        }

        for (std::vector<int>::size_type i = 0; i < input_topics.size(); i++)
        {
            if (topic.compare(input_topics[i]) == 0)
            {
```

```
                pcl_conversions::toPCL(tmpCloud2, clouds[i]);
                clouds_modified[i] = true;
            }
        }

        // Count how many scans we have
        std::vector<int>::size_type totalClouds = 0;
        for (std::vector<int>::size_type i = 0; i < clouds_modified.size(); i++)
        {
            if (clouds_modified[i])
            {
                totalClouds++;
            }
        }

        // Go ahead only if all subscribed scans have arrived
        if (totalClouds == clouds_modified.size())
        {
            pcl::PCLPointCloud2 merged_cloud = clouds[0];
            clouds_modified[0] = false;

            for (std::vector<int>::size_type i = 1; i < clouds_modified.size();
i++)
            {
#if PCL_VERSION_COMPARE(>=, 1, 10, 0)
                merged_cloud += clouds[i];
#else
                pcl::concatenatePointCloud(merged_cloud, clouds[i],
merged_cloud);
#endif

                clouds_modified[i] = false;
            }

            Eigen::MatrixXf points;

            pcl::getPointCloudAsEigen(merged_cloud, points);

            pointcloud_to_laserscan(points, &merged_cloud);

            // Publish point cloud after publishing laser scan as for some
reason moveFromPCL is causing getPointCloudAsEigen to
            // throw a segmentation fault crash
            std::shared_ptr<sensor_msgs::msg::PointCloud2> cloud_msg =
std::make_shared<sensor_msgs::msg::PointCloud2>();

            pcl_conversions::moveFromPCL(merged_cloud, *cloud_msg);

            point_cloud_publisher_->publish(*cloud_msg);
        }
    }

    void LaserscanMerger::pointcloud_to_laserscan(Eigen::MatrixXf points,
pcl::PCLPointCloud2 *merged_cloud)
    {
        sensor_msgs::msg::LaserScan output;
        output.header = pcl_conversions::fromPCL(merged_cloud->header);
        output.angle_min = this->angle_min;
```

```
        output.angle_max = this->angle_max;
        output.angle_increment = this->angle_increment;
        output.time_increment = this->time_increment;
        output.scan_time = this->scan_time;
        output.range_min = this->range_min;
        output.range_max = this->range_max;

        uint32_t ranges_size = std::ceil((output.angle_max - output.angle_min) /
output.angle_increment);
        output.ranges.assign(ranges_size,
std::numeric_limits<double>::infinity());

        for (int i = 0; i < points.cols(); i++)
        {
            const float &x = points(0, i);
            const float &y = points(1, i);
            const float &z = points(2, i);

            if (std::isnan(x) || std::isnan(y) || std::isnan(z))
            {
                RCLCPP_DEBUG(this->get_logger(), "rejected for nan in point(%f,
%f, %f)\n", x, y, z);
                continue;
            }

            double range_sq = pow(y, 2) + pow(x, 2);
            double range_min_sq_ = output.range_min * output.range_min;
            if (range_sq < range_min_sq_)
            {
                RCLCPP_DEBUG(this->get_logger(), "rejected for range %f below
minimum value %f. Point: (%f, %f, %f)", range_sq, range_min_sq_, x, y, z);
                continue;
            }

            double angle = atan2(y, x);
            if (angle < output.angle_min || angle > output.angle_max)
            {
                RCLCPP_DEBUG(this->get_logger(), "rejected for angle %f not in
range (%f, %f)\n", angle, output.angle_min, output.angle_max);
                continue;
            }

            int index = (angle - output.angle_min) / output.angle_increment;
            if (output.ranges[index] * output.ranges[index] > range_sq)
            {
                output.ranges[index] = sqrt(range_sq);
            }
        }

        laser_scan_publisher_->publish(output);
    }

}
```

- Lidar point cloud fusion configuration parameter file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/common_param.yaml
```

- The parameter meanings are as follows:
- `destination_frame`: The coordinate frame after Lidar fusion, here it is `base_link`.
- `cloud_destination_topic`: The topic for the fused Lidar point cloud data, here it is `/merged_cloud`.
- `scan_destination_topic`: The topic for the fused Lidar data, here it is `/scan_multi`.
- `laserscan_topics`: The Lidar topics to be fused.
- `angle_min` and `angle_max`: The minimum and maximum angles of the output scan, in radians. Here, it is from -3.14 to 3.14, indicating a 360-degree scan.
- `angle_increment`: The angular increment of the output scan, here it is 0.008726646 radians.
- `range_min` and `range_max`: The minimum and maximum distances of the output scan, in meters. Here, it is from 0.05m to 12.0m.

```
# Radar Fusion Node Configuration
laserscan_multi_merger:
  ros__parameters:
    destination_frame: "base_link"
    cloud_destination_topic: "/merged_cloud"
    scan_destination_topic: "/scan_multi"
    laserscan_topics: "/scan2 /scan3"
    angle_min: -3.14
    angle_max: 3.14
    angle_increment: 0.008726646  # Approximately 0.5 degrees in radians
    scan_time: 0.0
    range_min: 0.05
    range_max: 12.0
```

4.2 Point Cloud Filtering

- Point cloud filtering uses the binary package `laser_filters` and its `scan_to_scan_filter_chain` node, which can filter Lidar point cloud data in a specified region.

- Parameter file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/common_param.yaml
```

- Here, the Lidar point cloud is filtered to remove a rectangular box corresponding to the vehicle's body contour, with side lengths of 0.24 * 0.3 * 1.0m.
- See the comments below for the specific meaning of the parameters.

```
# Radar filter node configuration
laser_filter:
  ros__parameters:
    filter1:
      name: box_filter
      type: laser_filters/LaserScanBoxFilter
```

```
      params:
        box_frame: base_link  # The coordinate frame of the rectangular box
(usually the robot's base coordinate frame)
        min_x: -0.12           # Minimum x-value of the box
        max_x: 0.12            # Maximum x-value of the box
        min_y: -0.15           # Minimum y-value of the box
        max_y: 0.15            # Maximum y-value of the box
        min_z: -0.5            # Minimum z-value of the box (Lidar is usually
near z=0, so a negative value can be set)
        max_z: 0.5             # Maximum z-value of the box
        invert: false         # false = remove points inside the box; true =
keep points inside the box and remove points outside
```

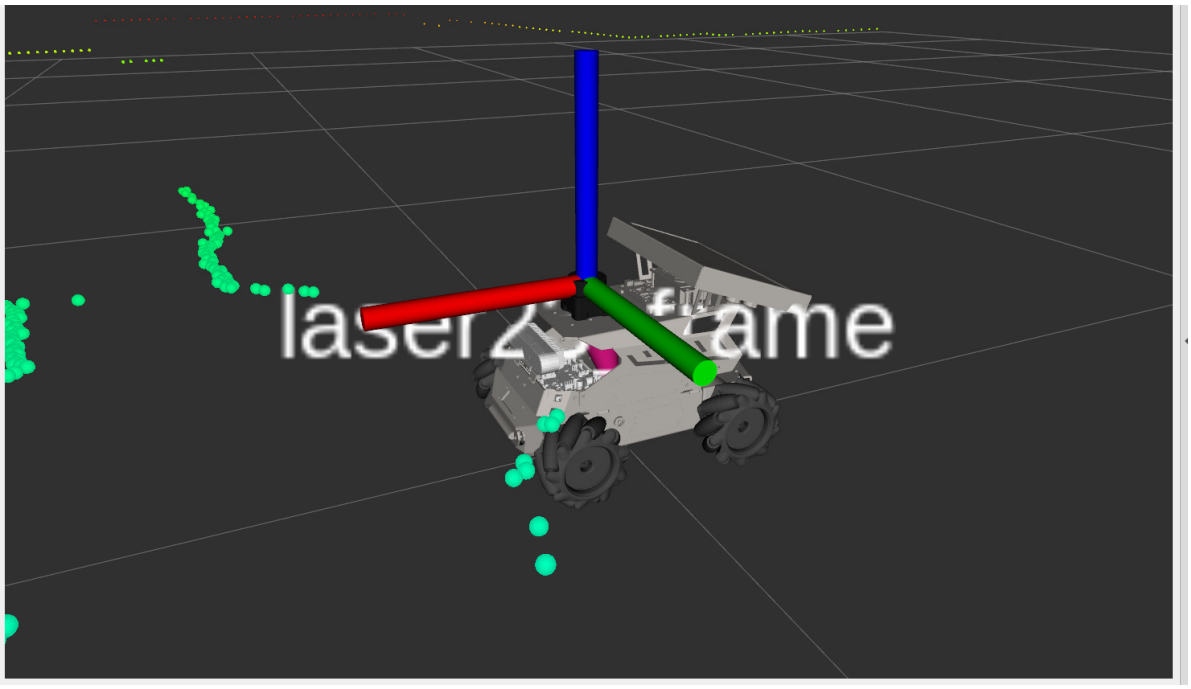
4.3 Lidar Launch File

- Source code path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/common/bringup_laser.launch.py
```

- Two launch methods are provided here: launching through a composite container and launching through independent nodes. The `use_composition` launch parameter is used to distinguish between them. By default, the independent node method is used.

- For the dual-Lidar version:
  - Since the installation position of the dual Lidars is lower on the z-axis than the single-Lidar version,
  - to maintain consistency, the fused Lidar will have a virtual root coordinate frame: `lasersim_frame`. The origin of this coordinate frame is the same as the Lidar coordinate frame of the single-Lidar version.



```
def generate_launch_description()->LaunchDescription:
    LASER_TYPE = os.environ.get('LASER_TYPE', 'single')

    # Different lidar topics are set according to LASER_TYPE.
    if LASER_TYPE == "dual":
        laserscan_topics = "/scan0 /scan1"
        multi_frame="lasersim_frame"
    elif LASER_TYPE == "single":
        laserscan_topics = "/scan2 /scan3"
        multi_frame="laser23_frame"
    else:
        raise ValueError(f"LASER_TYPE must be one of 'single' or 'dual', but got
'{LASER_TYPE}'")

    # Declare arguments
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "namespace",
            default_value="",
            description="Add namespace",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
                'use_composition',
                default_value="False",
                description='Whether to use composed bringup',
            )
    )

    # Initialize Arguments
    namespace=LaunchConfiguration("namespace")
    use_composition=LaunchConfiguration("use_composition")
    config_file = os.path.join(get_package_share_directory('m3_bringup'),
'config', 'common_param.yaml')

    # Topic Remapping
    remappings=[('/scan_filtered', [namespace, '/scan']), # The topic of LiDAR
after filtering
                ('/scan', [namespace, '/scan_multi'])]    # LiDAR topic before
filtering

    # Robot state publishing node, radar fusion and filtering must first publish
the robot model tf state
    robot_display=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(
            get_package_share_directory('m3_description'),
            'launch',
            'display.launch.py'
            )),
        launch_arguments={
            'prefix': namespace,
            'gui': 'False'  # Set to True only when the user is debugging alone;
otherwise, set to False.
        }.items(),
    )
```

```
    # Start the radar processing node in standalone mode
    load_nodes = GroupAction(
        condition=IfCondition(PythonExpression(['not ', use_composition])),
        actions=[
            # LiDAR data fusion
            PushRosNamespace(namespace=namespace),
            Node(
                package='laser_merge',
                executable='lasers_merger',
                name='laserscan_multi_merger',
                parameters=[config_file,
                            {'laserscan_topics': laserscan_topics},
                            {'destination_frame': multi_frame}
                           ],
            ),
            # LiDAR data filtering
            Node(
                package="laser_filters",
                executable="scan_to_scan_filter_chain",
                name='laser_filter',
                parameters=[config_file],
                remappings=remappings
            ),
            # Log Printing
            LogInfo(
                    msg=[Fore.GREEN+'LASER start sucefully!,Start by independent
node',Fore.RESET]
                )
        ]
    )

    # Start the radar processing node in a container manner
    load_composable_nodes=GroupAction(
        condition=IfCondition(use_composition),
        actions=[
            # PushRosNamespace(namespace=namespace),
            ComposableNodeContainer(
                    name='laser_process_container',
                    package='rclcpp_components',
                    namespace=namespace,
                    executable='component_container_isolated',
                    composable_node_descriptions=[
                        # LiDAR data fusion
                        ComposableNode(
                            package='laser_merge',
                            plugin='laserscanmerger::LaserscanMerger',
                            name='laserscan_multi_merger',
                            parameters=[config_file,
                                        {'laserscan_topics': laserscan_topics},
                                        {'destination_frame': multi_frame}
                                       ],
                            extra_arguments=[{'use_intra_process_comms': True}],
                        ),
                        # LiDAR data filtering
                        ComposableNode(
                            package='laser_filters',
                            plugin='ScanToScanFilterChain',
```

```
                            name='laser_filter',
                            parameters=[config_file],
                            remappings=remappings,
                            extra_arguments=[{'use_intra_process_comms': True}],
                        ),
                    ]
                ),

        # Log Printing
        LogInfo(msg=[Fore.GREEN+'LASER Start Sucefully!,Start Via
Container',Fore.RESET])
        ]
    )


    delay_start_composable_nodes= TimerAction(
        period=3.0,
        actions=[load_composable_nodes]
     )

    ld = []
    ld.extend(declared_arguments)
    ld.append(robot_display)
    ld.append(load_nodes)
    ld.append(delay_start_composable_nodes)

    return LaunchDescription(ld)
```